

Making symmetric block cipher meta permutating again

Heinrich Elsigan

January 27, 2026

Contents

1	Thanks to	1
2	Theory	2
2.1	8-staged symmetric block cipher pipeline	2
2.2	ZenMatrix a simplest possible algortihm to basically understand symmetric blockcipher	3
2.2.1	Mapping	4
3	Software	4
3.1	Git: PermAgainCrypt minimalistic repository	4
3.2	Download	4
3.3	Documentation online en-/decrypt form	4
3.4	Documentation of WinForm	6
3.5	Documentation of Windows Console	6
3.6	Java JFrame based	6
3.7	How to write Mathematics	6
3.8	Good luck!	8

Abstract

Hi, I'm [Heinrich Elsigan](#) I'm interested in CSharp, Java, MSSQL, .Net Core, Android and politics, society and the future; currently working as freelancer (one person company) and planning a secure endpoint 2 entpoint chat and looking to collaborate on reviews for other repositories and projects.

1. social media, development and other links:
2. personal tech and political blog blog.area23.at
3. GitHub repositories github.com/heinrichelsigan/
4. live demo .Net area23.at/net
5. StackOverflow stackoverflow.com/users/12213151/heinrich-elsigan
6. Curriculum vitae heinrichelsigan.area23.at/cv

1 Thanks to

Normally, thanks to are always at the end of each paper, but the people or organizations I benefited from while trying to make AES strong again are more important than a simple proof of concept showing that it works, except on my raw experimental test form.

1. <https://bouncycastle.org/>
2. <https://schneier.com/>
3. <https://github.com/dotnet/>
4. <https://github.com/microsoft>
5. <https://git.lysator.liu.se/nettle/nettle> (real easy to read c/c++ code)

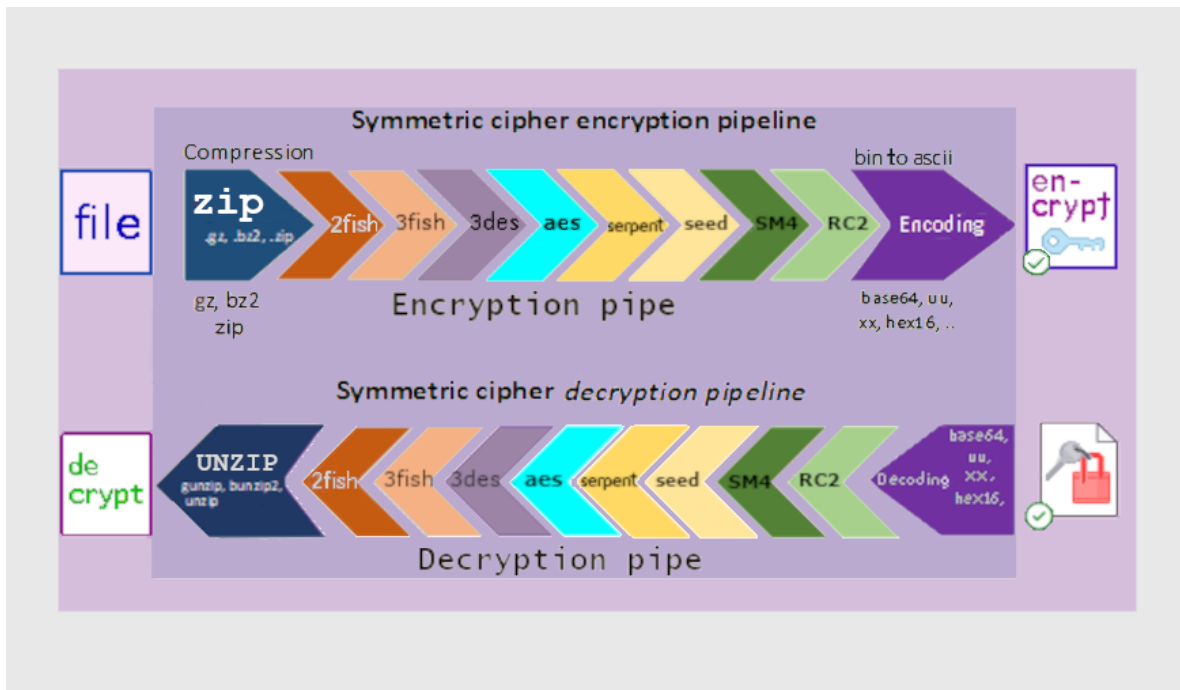


Figure 1: Cipher Pipeline

2 Theory

2.1 8-staged symmetric block cipher pipeline

An eight staged symmetric block cipher crypto pipeline to improve advanced encryption standard based on meta DES, 3DES with P-Box S-Box.

The following image shows you an example of a symmetric cipher 8 staged encryption pipe and the corresponding decryption inverse pipe.

Before entering the encryption pipe, the file can be zipped to avoid huge amount of symmetric cipher blocks and after exiting the encryption pipe the file can be ascii encoded with base64 mime, uuencode, xxencode or hex16, because symmetric chiphered binary files might lose their block padding.

Implementation is based on my blog article: [Making symmetric cipher encryption meta permutating again](#), including the follwing symmetric cipher algorithms:

...or bullet points ...

- [Aes](#), [AesLight](#), [Rijndael](#)
- [Bruce Schneier's](#) BlowFish, 2-Fish, 3-Fish
- [Camellia](#), [CamelliaLight](#)
- [Cast5](#), [Cast6](#)
- [National security agency's](#) Des, 3-Des, [SkipJack](#)
- [Dstu7624](#)
- [Ghost](#), [Idea](#), [Noekeon](#)
- [RC2](#), [RC532](#), [RC564](#), [RC6](#)
- [SEED](#), [SM4](#)
- [Serpent](#), [Tnepres](#)
- [Tea](#), [XTea](#)
- and my own simplest symmetric block cipher algoithms: [ZenMatrix](#), [ZenMatrix2](#)

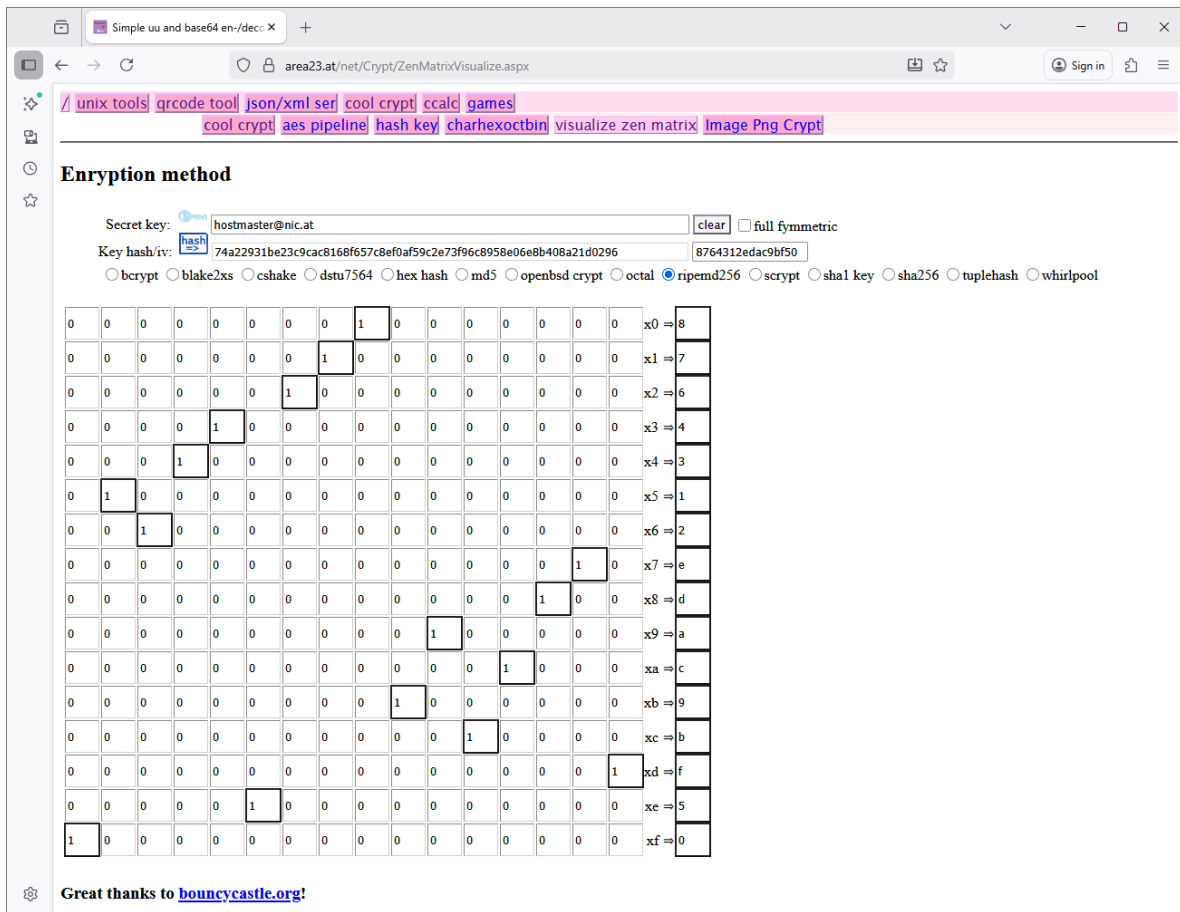


Figure 2: ZenMatrix Visualize

2.2 ZenMatrix a simplest possible algortihm to basically understand symmetric blockcipher

Starting from a no permutating 1-matrix, where projection is same as base, ZenMatrix generates a permutating mapping with blocksize 16 a matrix with only one 1 per row and column (rest is 0) to change position inside block and value offset.

<https://area23.at/net/Crypt/ZenMatrixVisualize.aspx>

x	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
x0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
x1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
x2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
x3	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
x4	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
x5	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
x6	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
x7	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
x8	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
x9	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
xA	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
xB	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
xC	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
xD	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
xE	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
xF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

When entering hostmaster@nic.at with hash ripemd256 and not fully symmetric checked, the matrix will look like this:

x	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF	Mx
x0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	8
x1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	7
x2	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	6
x3	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	4
x4	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	3
x5	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
x6	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2
x7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	E
x8	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	D
x9	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	A
xA	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	C
xB	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	9
xC	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	B
xD	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	F
xE	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	5
xF	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

2.2.1 Mapping

This mean that a byte[8] e.g. 321F 0000 0000 0000 $x0 \implies x8, x1 \implies x7, x2 \implies x6, x3 \implies x4, xF \implies x0$, is mapped to 8880 8868 4888 8888.

3 Software

Source code of PermAgainCrypt is hosted at Github. Github releases contains compiled and linked .exe files for windows, but most users prefer download from a separate download site <https://cqrxs.eu/>.

3.1 Git: PermAgainCrypt minimalistic repository

<https://github.com/heinrichelsigan/PermAgainCrypt/>

3.2 Download

Go to <https://cqrxs.eu/> or <https://io.cqrxs.eu/> and choose latest version. You can directly go to the download area <https://cqrxs.eu/download/> or <https://io.cqrxs.eu/download/>. Attention, version before March 2025 have a 3-fish over Aes-Engine encryption bug, because 3-fish uses AES default block size and key length and Bouncy Castle Aes-Engine parameters. It works, but settings AES default engine for 3-fish, isn't so serious and please download version after March 30.

3.3 Documentation online en-/decrypt form

1. Key: When clicking key your entered key will be stored temporary in session.
2. Textbox secret key: Enter your Email address and secret key.
3. Buton Clear: Clear and reset the entire form.
4. Hyperlink Help: Show this help
5. RadioButtonList KeyHashes Choose the hash method to hash your secret key.
6. ImageButton Hash: Clicking will hash your key and display hashed key in textbox.
7. TextBox Hash (readonly): Displays your hashed key.
8. Button "Set Pipe": sets symmetric cipher pipe, dependent only on your entered key

Encryption method

1. Key: 3. 4. [Help](#)

5. ☐ bcrypt ☐ blake2xs ☐ cshake ☐ dstu7564 ☒ hex ☐ md5 ☐ openssh ☐ octal ☐ ripemd256 ☐ scrypt ☐ sha1 ☐ sha256 ☐ tuplehash ☐ whirlpool

6. hash 7. 8. 9.

10. 11. 12. 13.

Hint: zip and 7zip compression are still buggy implemented, please use only bzip2 and gzip.

En-/Decrypt file

☒ encode file 14. 15. 16.

En-/Decrypt text

17. 18.

19. 20. 21.

Great thanks to [bouncycastle.org!](#)

Figure 3: Symmetric Cipher WebForm

9. Button “Hash Pipe”: sets symmetric cipher pipe, dependent primary on calculated hash and secondary on your entered key.
10. DropDown ZipTypes Choose encryption type (Please only GZip or Zip or None)
11. DropDown CipherTypes Choose a symmetric cipher to add it to the symmetric cipher pipe.
12. ImageButton „add algo“: Clicking on will add the in c. DropDown CipherTypes selected symmetric cipher algorithm to CipherPipe.
13. TextBox CipherPipe (readonly): Displays the current Cipher Pipe algorithms.
14. ImageButton “Clear Pipe”: Clicking on will clear only the entire Cipher Pipe
15. DropDown EncodingTypes: Choose the final binary to ascii encoder, default is Base64. Beware of using uuencode in Web, because `j` will be interpreted as possible html injection.
16. TextArea Source: paste or enter Text here.
17. TextArea Destination: (readonly) After clicking Encrypt or Decrypt processed text will appear in the text area.
18. Button Encrypt: Encrypts the text from TextArea Source and displays encrypted text in TextArea Destination.
19. Button “Random Text”: Adds a short fortune to TextArea Source.
20. Button Decrypt: Decrypts text in TextArea Source and display decrypted text in TextArea Destination.

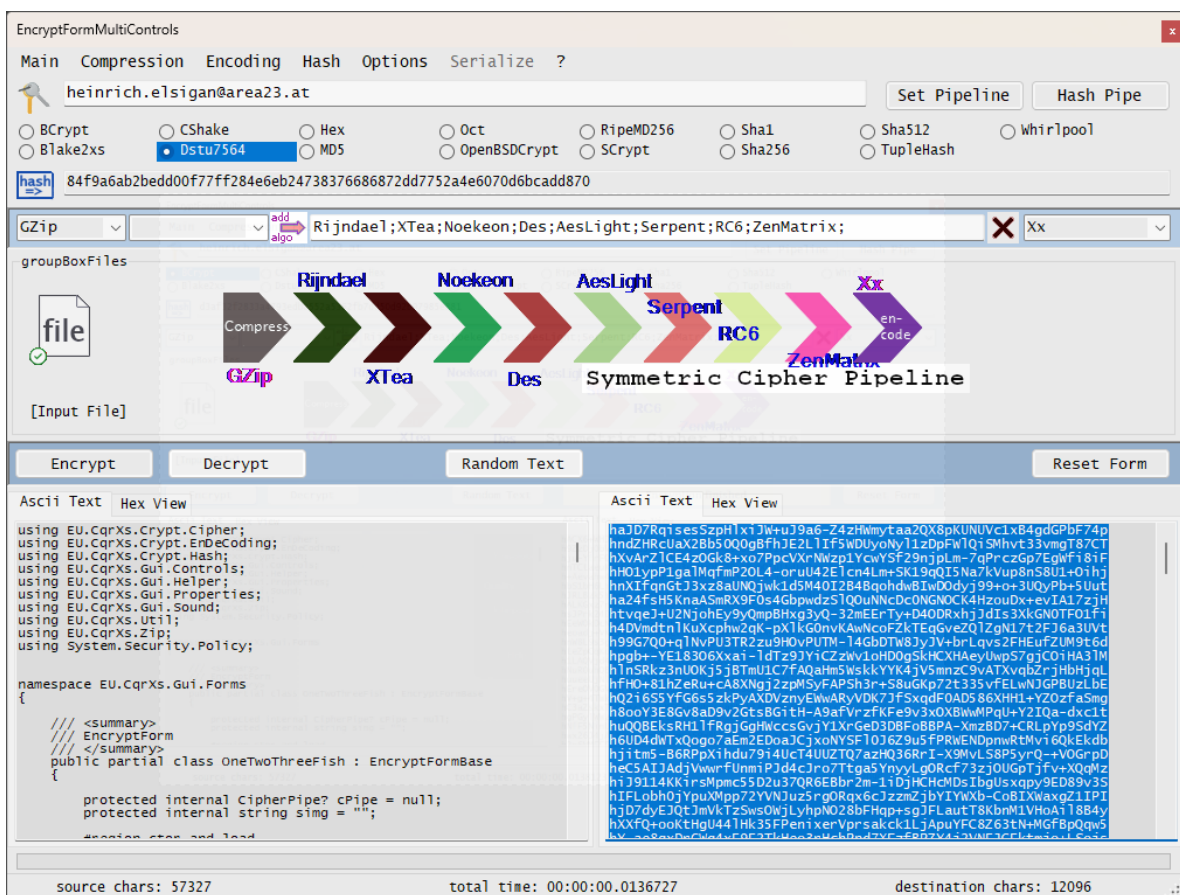


Figure 4: WinForm CSharp

3.4 Documentation of WinForm

You can also go directly to github.com/heinrichelsigan/PermAgainCrypt/releases to download <https://github.com/heinrichelsigan/PermAgainCrypt/releases/download/2026-01-25-Hex64-Console-JFrame-Latex-release-v2.26.125/2026-01-25-EU.CqrXs.Gui.x64.7z> ...

3.5 Documentation of Windows Console

Download the latest Console from io.cqrxs.eu/download/EU.CqrXs.Console/

3.6 Java JFrame based

- <https://docs.oracle.com/javase/8/docs/api/javax/swing/JFrame.html>
- <https://github.com/openjdk-mirror/jdk7u-jdk/blob/master/src/share/classes/javax/swing/JFrame.java>

3.7 How to write Mathematics

$\text{\LaTeX}$ is great at typesetting mathematics. Let $X_1, X_2, \dots, X_n$ be a sequence of independent and identically distributed random variables with $E[X_i] = \mu$ and $\text{Var}[X_i] = \sigma^2 < \infty$, and let

$$S_n = \frac{X_1 + X_2 + \dots + X_n}{n} = \frac{1}{n} \sum_{i=1}^n X_i$$

denote their mean. Then as n approaches infinity, the random variables $\sqrt{n}(S_n - \mu)$ converge in distribution to a normal $\mathcal{N}(0, \sigma^2)$.


```

C:\Users\heinrich.elsigan\source\PermAgainCrypt\Deploy\console\64\EU.CqrXs.Console.exe -7
unrecognized option: -7.
Usage: EU.CqrXs.Console.exe
-i | --infile= | --inText={string|EnvironmentVariable} | --inStd
|
-k | --keypassKey encrypt
-H | --Hash={Oct|Blake2xs|BCrypt|CShake|Dstu7664|MD5|RipeMD256|SCrypt|Sha1|Sha256|Sha384|Sha512|Whirlpool|TupleHash}
-z | --zip={gzip|bzip2|zip}
-C | --CipherAlgost={algol, algo2, ...}
  algo:
    Aes, AesLight, Rijndael, Des, Des3, Dstu7624,
    Aria, Camellia, CamelliaLight, Cast5, Cast6,
    BlowFish, Fish2, Fish3,
    Gost28147, Idea, Noekeon,
    RC2, RC532, RC564, RC6,
    Seed, SkipJack, Serpent, SM4,
    Tea, Tnepres, XTea,
    ZenMatrix, ZenMatrix2
  symmAlgo:
    Aes, BlowFish, Camellia, Cast6, Des3, Fish2, Fish3, Gost28147, Idea, RC532, Seed, SkipJack, Serpent, Tea, XTea, SM4
-S | --SymmCipher
-e | --encode={raw|hex16|hex32|base32|base64|uu}
-D | --Decrypt=Inverse_Pipe_Direction
|
-o | --outfile= | --outText=EnvironmentVariable | --outStd
|
-Y | --YankeeTest
-? | --gethelp

Examples:
EU.CqrXs.Console.exe -i=. \README.MD -e=base16 -o=. \README.MD.base16
EU.CqrXs.Console.exe -D -i=. \README.MD.base16 -e=base16 -o=. \README.MD.txt

EU.CqrXs.Console.exe -i=. \README.MD -k=Hallo -z=gzip -C=BlowFish, Fish2, Fish3 -e=base64 -o=. \README.MD.gz.BFF.base64
EU.CqrXs.Console.exe -D -i=. \README.MD.gz.BFF.base64 -e=base64 -C=BlowFish, Fish2, Fish3 -p=Hallo -z=gzip -o=. \README.GUNZIP.txt

EU.CqrXs.Console.exe -i=. \README.MD -z=bz -k=heinrich.elsigan.area23.at -H=Whirlpool -e=hex32 -o=. \README.MD.Whirlpool.bz.Hex32
EU.CqrXs.Console.exe -D -i=. \README.MD.Whirlpool.bz.Hex32 -e=hex32 -k=heinrich.elsigan.area23.at -H=Whirlpool -z=bz -o=. \README.BUNZIP.txt

EU.CqrXs.Console.exe -i=. \README.MD -z=zip -k=io.cqrxs.eu -C=Aes, Blowfish, Des3, Fish2, Fish3, Seed, Serpent, SM4 -H=SCrypt -e=uu -o=. \README.MD.SCrypt.zip.uu
EU.CqrXs.Console.exe -D -i=. \README.MD.SCrypt.zip.uu -e=uu -k=io.cqrxs.eu -C=Aes, Blowfish, Des3, Fish2, Fish3, Seed, Serpent, SM4 -H=SCrypt -z=zip -o=. \README.UNZIP.txt

EU.CqrXs.Console.exe -i=. \README.MD -S -z=zip -k=io.cqrxs.eu -H=BCrypt -e=xx -o=. \README.MD.BCrypt.zip.xx
EU.CqrXs.Console.exe -D -i=. \README.MD.BCrypt.zip.xx -S -e=xx -k=io.cqrxs.eu -H=BCrypt -z=zip -o=. \README.SYM_BCRYPT_UNZIP.txt\...\n

C:\Users\heinrich.elsigan\source\PermAgainCrypt\Deploy\console\64\

```

Figure 5: Windows EU.CqrXs.Console.exe

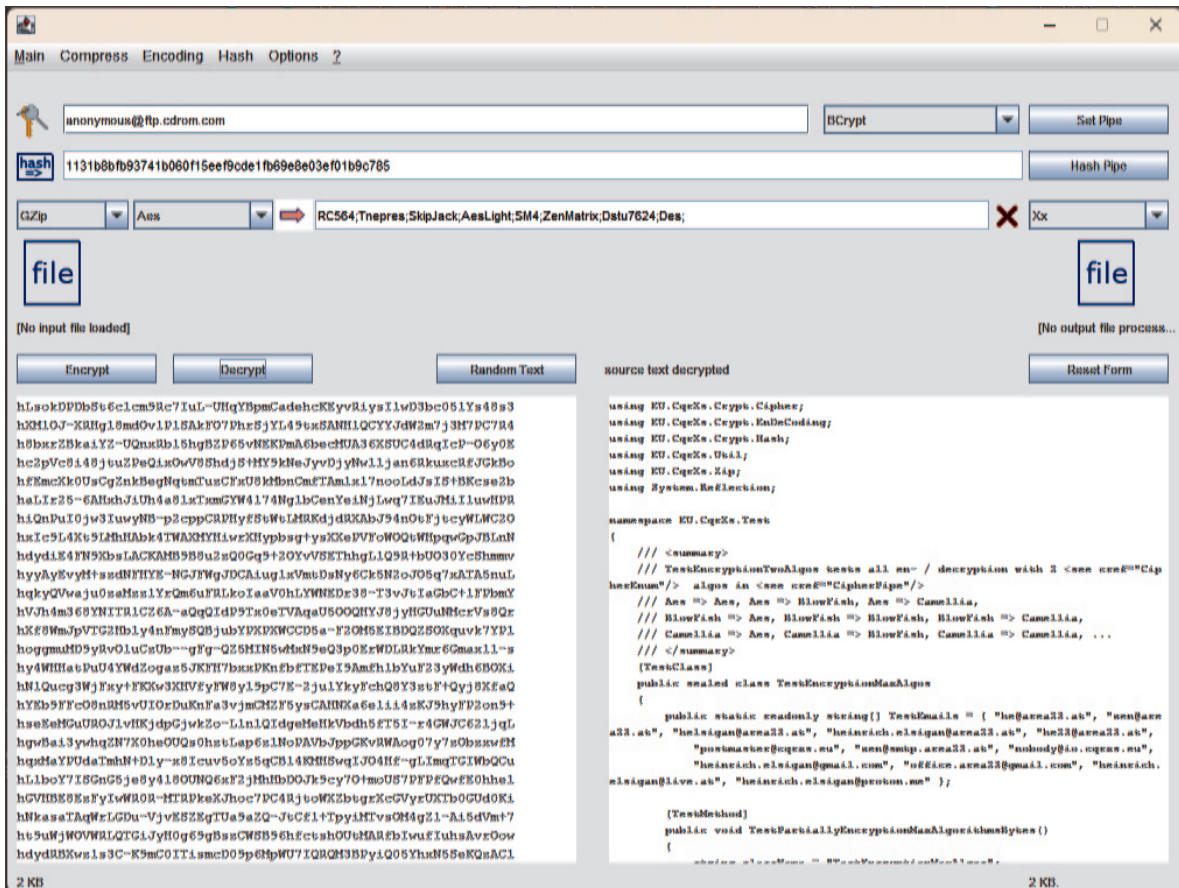


Figure 6: javax.swing.JFrame.java

3.8 Good luck!

We hope you find PermAgainCrypt useful, and good luck. [help library](#) for more tutorials and user guides! Please also let us know if you have any feedback using the **Contact us** link at the bottom of the Overleaf menu — or use the contact form at <https://heinrichelsigan.area23.at>.

References